



C. F. G. S. Desarrollo de Aplicaciones Web - Distancia -

Examen de Programación de la 2ª Evaluación. 24-febrero-2026

“First, solve the problem. Then, write the code.” - John Johnson –

1) Se desea desarrollar una aplicación para gestionar información de atletas.

Programar una clase llamada Deportista, que esté en un paquete de nombre “deportes”, con las siguientes propiedades:

- nombre (String)
- edad (int)
- peso (double) en kg
- altura (double) en metros
- horasEntrenamientoSemanal (int)

La clase tendrá los siguientes métodos:

- a. Constructor por defecto que no haga nada.
- b. Constructor al que se le pasan como argumento valores para todas las propiedades de la clase.
- c. Métodos set y get para todas las propiedades de la clase.
- d. Método calcularIndiceRendimiento() que calcule y devuelva el siguiente valor:

$$IR = \frac{horasEntrenamientoSemanal}{peso * altura}$$

- e. Método clasificarRendimiento() que:
 - Devuelva "Bajo" si IR < 0.5
 - Devuelva "Medio" si IR está entre 0.5 y 1 (incluido)
 - Devuelva "Alto" si IR > 1

2) Para probar el funcionamiento de la clase Deportista, hacer un programa en el que se declare y se le den valores a un array de 10 objetos de la clase Deportista. El programa estará en un paquete de nombre “principal”, y se le darán valores a los objetos del array utilizando los siguientes criterios:

- Nombre: Deportista1, Deportista2, Deportista3,, Deportista10
- Edad: Valor aleatorio comprendido entre 16 y 40 años, ambos incluidos.
- Peso: Valor aleatorio comprendido entre 40 y 120 kg, ambos incluidos.
- Altura: Valor aleatorio comprendido entre 1.50 y 2.10 metros.
- Horas de entrenamiento semanal:
Valor aleatorio comprendido entre 1 y 20 horas, ambos incluidos.

Escribir a continuación los datos de los 10 participantes, con el siguiente formato:

Ejemplo de ejecución del programa:

----- DEPORTISTA 1 -----



Nombre: Deportista1
Edad: 24 años
Peso: 78.0 kg
Altura: 1.82 m
Horas entrenamiento semanal: 12
Índice de Rendimiento: 0.084
Clasificación: Bajo

----- DEPORTISTA 2 -----

Nombre: Deportista2
Edad: 19 años
Peso: 65.0 kg
Altura: 1.70 m
Horas entrenamiento semanal: 18
Índice de Rendimiento: 0.163
Clasificación: Bajo

.....
.....

- 3) Hacer un programa en el que se generen 20 valores aleatorios comprendidos entre 1 y 10, ambos incluidos. Los valores deberán guardarse en una tabla de 2 filas y 10 columnas.

Posteriormente, el programa deberá:

1. Mostrar la tabla generada.
2. Contar cuántas veces aparece cada número del 1 al 10 en la tabla, indicando número y frecuencia, utilizando el siguiente formato:

Tabla generada:

3 7 1 5 3 9 2 7 10 1
4 6 8 2 3 5 7 9 1 4

Número Frecuencia

1	3
2	2
3	3
4	2
5	2
6	1
7	3
8	1
9	2
10	1

- 4) Hacer un programa que pida por teclado un nombre completo, formado por un nombre y dos apellidos, separados por un único espacio entre cada palabra. A continuación, generar un nombre de usuario usando las siguientes reglas:



- Se toma la primera letra del nombre.
- Se toma la primera letra del primer apellido.
- Se concatena con el segundo apellido completo.
- Todo el nombre de usuario se convierte a minúsculas.

Ejemplo de ejecución del programa:

Ingrese su nombre completo (Nombre PrimerApellido SegundoApellido): Marisa Mesa Cid

Nombre completo: Marisa Mesa Cid

Nombre de usuario: mmcid

Puntuación de cada ejercicio:

Ejercicio 1:	3 puntos
Ejercicio 2:	3 puntos
Ejercicio 3:	2,5 puntos
Ejercicio 4:	1,5 puntos

Criterios de corrección y cómo entregar el examen:

Aunque un programa no funcione, entregarlo igual, ya que puede ocurrir que esté bien planteado y falle por un pequeño detalle. De la misma manera, si se pide hacer un programa con una determinada sentencia para implementar un bucle y no os sale, hacerlo de otra manera. Evidentemente no puntuará lo mismo, pero al menos podéis completar el ejercicio.

Hay que subir la solución del examen como tarea, en formato de procesador de texto en un único documento. El documento tiene que incluir el código fuente completo de cada uno de los ejercicios. Nombrar al documento con vuestro apellido y nombre.



AYUDA PARA EL EXAMEN

A la hora de programar, netbeans nos ayuda a escribir código de manera más rápida mediante decenas de atajos (shortcuts). NetBeans incluye un archivo pdf con la lista completa de dichos atajos. Esta lista se encuentra dentro del directorio de instalación de NetBeans en una carpeta llamada "nb". Allí encontraremos 2 pdfs:

shortcuts.pdf

shortcuts_mac.pdf

El primero contiene los atajos para PC (Windows/Linux), el segundo para los usuarios de Mac.

Acordaros dónde está el fichero shortcuts.pdf para poder consultarlo el día del examen. Os recomiendo que os familiaricéis con el uso de estos atajos. De todas maneras, os pongo a continuación algunos de los atajos más útiles:

1. sout + Tab, genera la línea: `System.out.println("");`
2. psvm genera un método main

```
public static void main(String[] args) {  
    }  
}
```

3. sw + Tab, genera una sentencia switch

```
switch (var) {  
    case val:  
  
        break;  
    default:  
        throw new AssertionError();  
}
```

4. for, while, if +ctrl+spacio dan sugerencias para construir cualquiera de estas estructuras de control

Tampoco es necesario saberse de memoria todos los métodos de cada una de las clases que tiene el lenguaje. Para eso podemos consultar la API del lenguaje y los IDEs nos ayudan. Si no recordamos como se calcula el valor absoluto de un valor, simplemente escribiendo `Math`. (clase utilizada en los apuntes), nos aparecen todos los métodos de la clase, con lo que podemos localizar el que nos interesa utilizar. De la misma manera, si después de una variable de tipo `String` pongo el punto también me aparecen todos los métodos (extraer un carácter, convertir a mayúsculas.....).



Math.

E	double
PI	double
IEEEremainder (double d, double dl)	double
abs (double d)	double
abs (float f)	float
abs (int i)	int
abs (long l)	long
acos (double d)	double
asin (double d)	double
atan (double d)	double
atan2 (double d, double dl)	double
cbrt (double d)	double
ceil (double d)	double
cos (double d)	double

String nombre;

nombre.

charAt (int i)	char
codePointAt (int i)	int
codePointBefore (int i)	int
codePointCount (int i, int il)	int
compareTo (String string)	int
compareToIgnoreCase (String string)	int
concat (String string)	String
contains (CharSequence cs)	boolean
contentEquals (CharSequence cs)	boolean

Plantilla de un pequeño programa que lee un valor por teclado.

```
import java.util.Scanner;
```

```
public class Plantilla {
```

```
    public static void main(String[] args) {  
        Scanner teclado = new Scanner(System.in);  
        int numero;
```

```
        System.out.println("Introduce un primer número:");  
        numero = teclado.nextInt();
```

```
    }  
}
```